TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Ana Lucia Pinargote Macias

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: #1: ESTUDIO DE CASO-SISTEMA DE CPNTROL RESCATE Y ADOPCION ANIMAL

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad __1__

Actividad # 1_ESTUDIO DE CASO-SISTEMA DE CONTROL DE RESCATE Y ADOPCION

Actualmente, los refugios y organizaciones que ayudan a los animales necesitan llevar un buen control de los rescates, la atención veterinaria y las adopciones. Cuando los datos se registran manualmente o están en diferentes documentos, es más difícil conocer la situación de cada animal y dar seguimiento a los casos pendientes.

Por eso, se propone crear un Sistema de Control de Rescate y Adopción Animal que permita registrar la información de los animales rescatados, organizar los casos y asignar voluntarios o veterinarios para su atención. También permitirá actualizar el estado de cada animal desde su rescate hasta su recuperación y posible adopción.

El sistema será utilizado por ciudadanos, voluntarios, veterinarios y administradores del refugio. Cada uno tendrá funciones específicas según su responsabilidad, facilitando la organización de la información y el seguimiento de las actividades.

Para diseñar el sistema se utilizarán diagramas UML, patrones de diseño y GitHub con la metodología Gitflow. Esto ayudará a organizar el desarrollo del proyecto y facilitará su mantenimiento y futuras mejoras.

Objetivo general

Diseñar la estructura de un Sistema de Control de Rescate y Adopción Animal que permita registrar los rescates, clasificar a los animales, asignar responsables y dar seguimiento a su atención y adopción, utilizando diagramas UML, patrones de diseño y GitHub con Gitflow.

Objetivo especifico

- ✓ Identificar a los usuarios del sistema y las funciones que realizará cada uno.
- ✓ Establecer los requerimientos necesarios para el funcionamiento del sistema.
- ✓ Elaborar los diagramas UML de casos de uso, clases y secuencia.
- ✓ Aplicar patrones de diseño para organizar y facilitar el mantenimiento del software.
- ✓ Utilizar GitHub y Gitflow para organizar el trabajo y guardar los avances del proyecto.

Descripción de la solución propuesta

El sistema permitirá registrar a los animales rescatados, incluyendo su especie, fecha y lugar del rescate, así como su estado actual. También facilitará la asignación de voluntarios y veterinarios para atender cada caso.

Los animales podrán pasar por diferentes estados: Rescatado, En Tratamiento, En Recuperación, Disponible para Adopción y Adoptado. Además, se podrá consultar su información y actualizar los cambios durante el proceso.

También se incluirán notificaciones para informar a los voluntarios sobre nuevos rescates. Para el diseño se aplicarán los patrones Singleton, Observer, Factory Method y MVC, y se utilizará GitHub con Gitflow para organizar el desarrollo y controlar los cambios del proyecto.

Identificación de los actores.

Los actores son las personas que interactúan con el sistema y realizan diferentes actividades según sus responsabilidades.

Actor	Descripción y funciones
Ciudadano o Deportante	Reporta animales que se encuentran abandonados, heridos o en situación de riesgo y proporciona la información necesaria para atender el caso
Voluntario	Colabora en los rescates consulta los casos asignados y actualiza la información de las actividades que realiza
Veterinario	Registra las evaluaciones medicas los tratamientos y la evolución del estado de salud animal
Administrador de refugio	Registra y supervisa los casos, asigna responsables administra la información y controla los procesos de adopción.

Requerimientos funcionales.

Los requerimientos funcionales describen las acciones que debe realizar el sistema para cumplir con sus objetivos.

Código	**Requerimiento funcional**	**Descripción**
RF01	**Registrar rescates**	Registrar la fecha, ubicación y descripción del rescate.
RF02	**Registrar animales**	Ingresar el nombre, edad aproximada, especie y características del animal.
RF03	**Clasificar especies**	Clasificar a los animales en perros, gatos o fauna silvestre

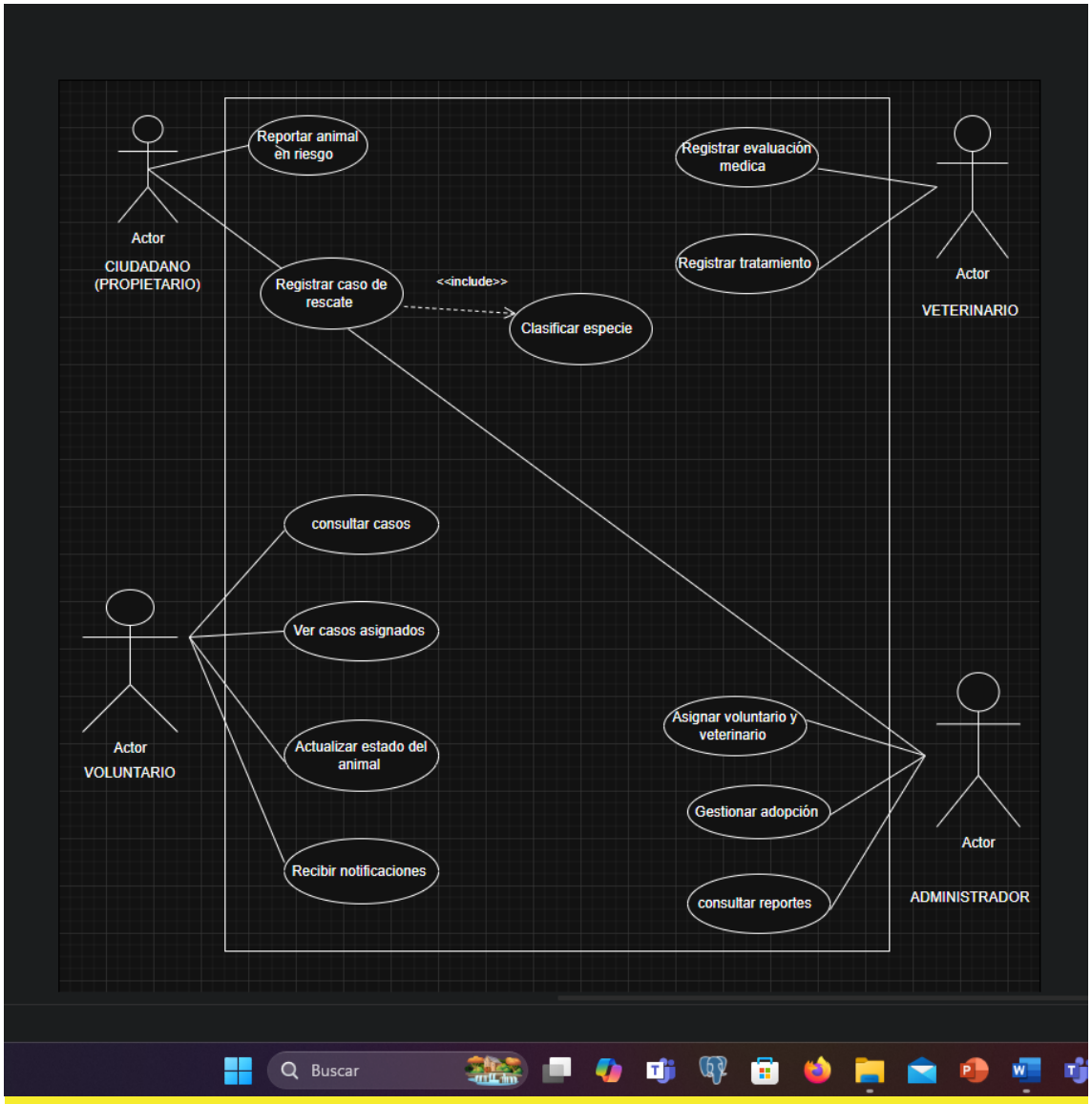
RF04	**Asignar responsables**	Permitir al administrador asignar voluntarios y veterinarios.
RF05	**Actualizar estados**	Cambiar el estado del animal según su situación.
RF06	**Registrar atención veterinaria**	Guardar evaluaciones, tratamientos y observaciones médicas.
RF07	**Gestionar adopciones**	Registrar solicitudes y actualizar el resultado de las adopciones.
RF08	**Consultar casos**	Permitir consultar la información de los casos según los permisos del usuario.
RF09	**Enviar notificaciones**	Notificar a los voluntarios sobre nuevos casos de rescate.
RF10	**Mantener historial**	Guardar un registro de los cambios realizados en cada caso.

Requerimientos no funcionales.

Código	**Requerimiento no funcional**	**Descripción**
RNF01	**Usabilidad**	La interfaz debe ser sencilla y fácil de utilizar
RNF02	**Seguridad**	El acceso a la información dependerá del tipo de usuario y sus permisos
RNF03	**Mantenibilidad**	El diseño debe facilitar las modificaciones y mejoras del sistema
RNF04	**Escalabilidad**	El sistema debe permitir agregar nuevas funciones y registrar mas animales

RNF05	**Trazabilidad**	Los cambios del proyecto deben quedar en GitHub mediante Git
RNF06	**Reutilización**	Los componentes deben facilitar el uso de soluciones en diferentes partes del sistema
RNF07	**Integridad de la información**	Los datos de los animales y sus casos deben mantenerse completos y coherentes

SISTEMAS DE CONTROL DE RESCATE Y ADOPCION ANIMAL



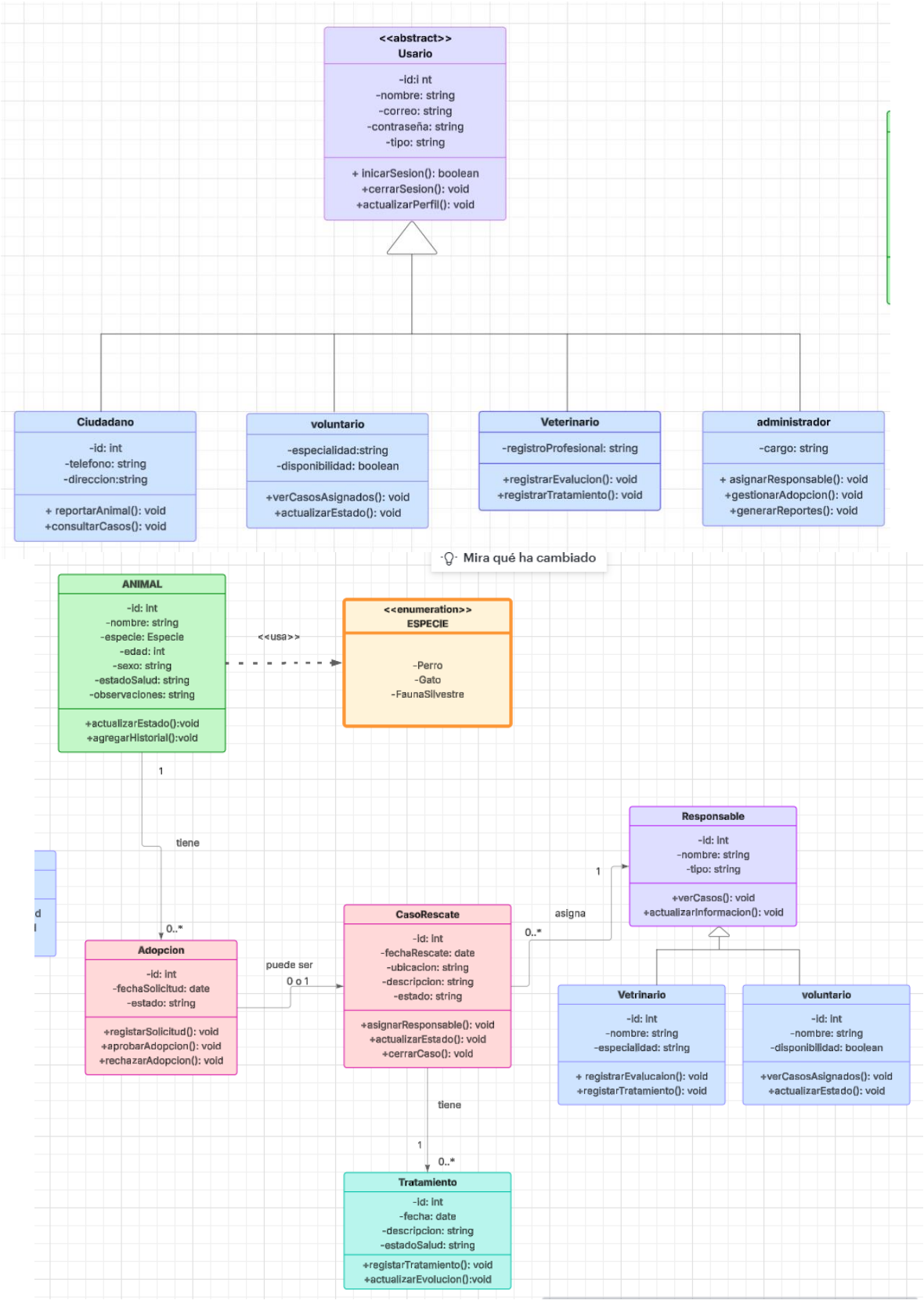
Aplicación de los principios SOLID en el diagrama de clases.

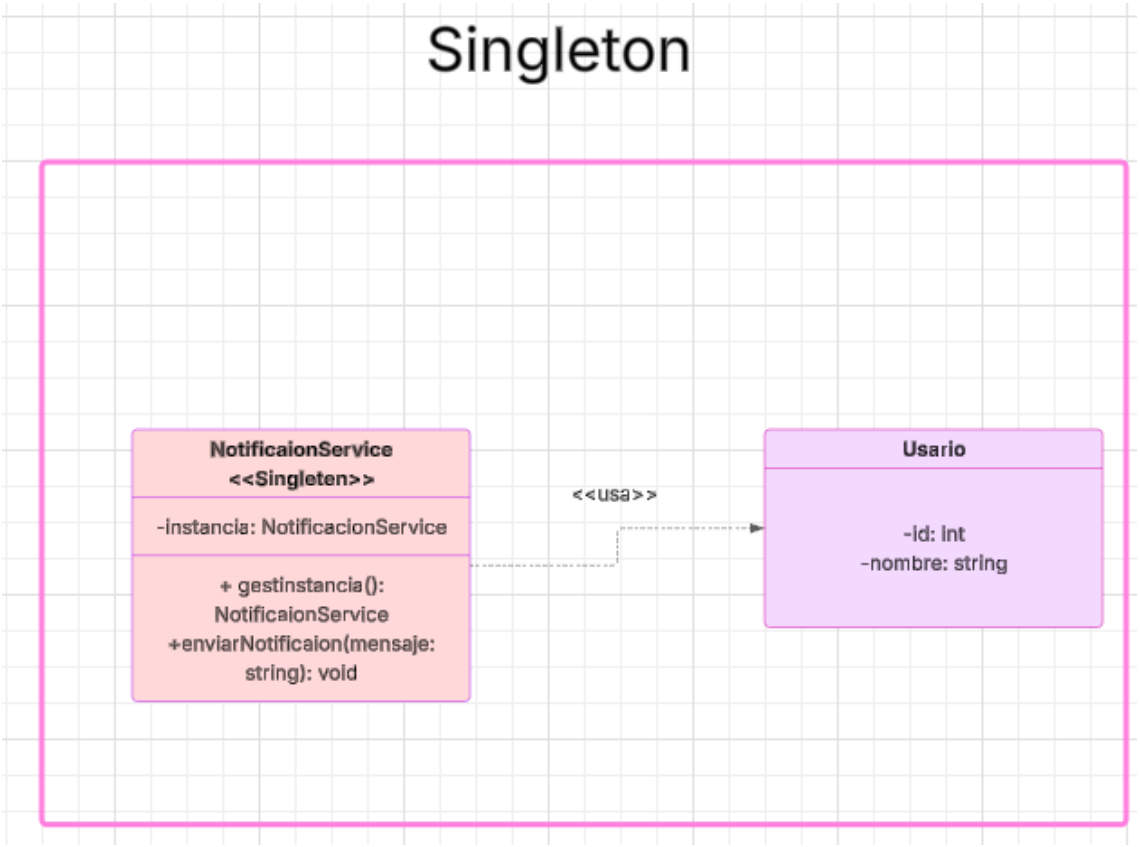
Este diagrama representa la estructura del Sistema de Control de Rescate y Adopción Animal mediante clases que organizan la información de los usuarios, animales, entre estas clases, permitiendo comprender cómo se organiza el sistema y facilitando su mantenimiento y futuras mejoras.

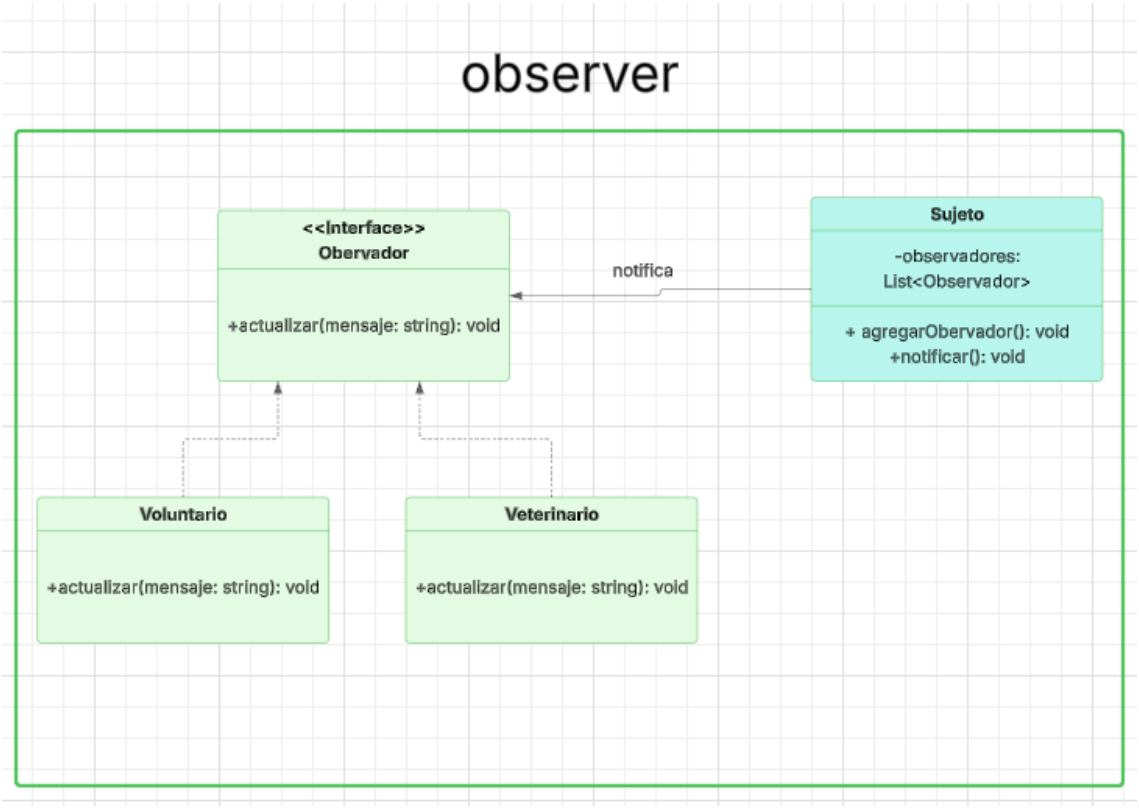
Asimismo, este diseño estructural se encuentra alineado bajo los principios SOLID, garantizando los siguientes aspectos:

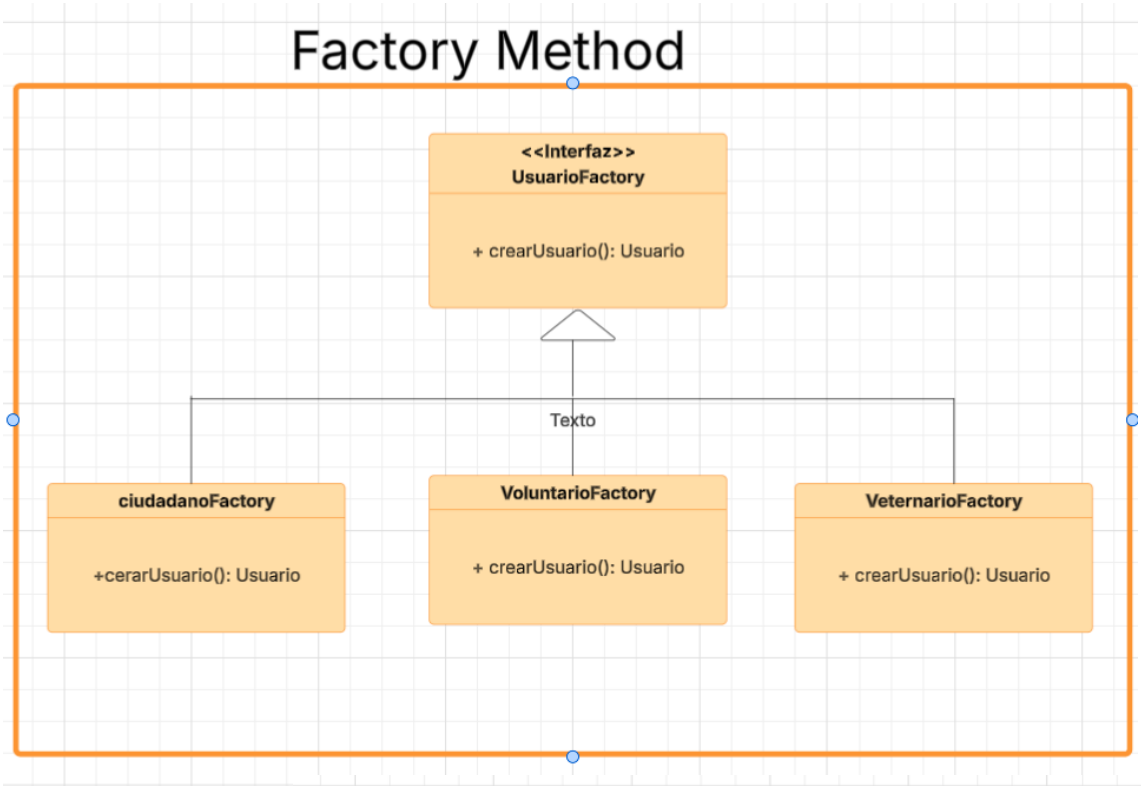
- ✓ **Principio de Responsabilidad Única (SRP):** Cada clase del sistema posee una única y bien definida razón para cambiar. Por ejemplo, la clase Animal se encarga de guardar y gestionar la información del animal y su estado físico, mientras que la clase CasoRescate maneja la ubicación y el seguimiento del rescate. De esta manera, cada clase cumple una función diferente y el sistema se mantiene más organizado y fácil de modificar.
- ✓ **Principio de Abierto/Cerrado (OCP):** Las clases e interfaces del sistema permiten agregar nuevas funciones sin tener que modificar el código que ya está funcionando. Por ejemplo, la clase abstracta Usuario permite crear nuevos tipos de usuarios, como otros roles que se necesiten en el futuro, aprovechando sus características principales sin cambiar la clase original.
- ✓ **Principio de Sustitución de Liskov (LSP):** Las clases que heredan de Usuario, como Ciudadano, Voluntario, Veterinario y Administrador, pueden utilizarse en lugar de la clase principal sin causar problemas en el sistema. Así, cada tipo de usuario puede cumplir sus funciones sin afectar el inicio de sesión ni la gestión de la información.
- ✓ **Principio de Segregación de Interfaces (ISP):** Al utilizar interfaces específicas, como las de los patrones Observer y Factory Method, cada clase trabaja únicamente con los métodos que necesita. Esto permite mantener el código más ordenado, evitar dependencias innecesarias y facilitar el funcionamiento de cada componente del sistema.
- ✓ **Principio de Inversión de Dependencias (DIP):** El diseño permite que los módulos del sistema trabajen de forma independiente mediante el uso de interfaces y abstracciones. Por ejemplo, las fábricas de creación y el sistema de notificaciones utilizan reglas generales en lugar de depender de clases específicas. Esto facilita realizar cambios y agregar nuevas funciones sin afectar el resto del sistema.

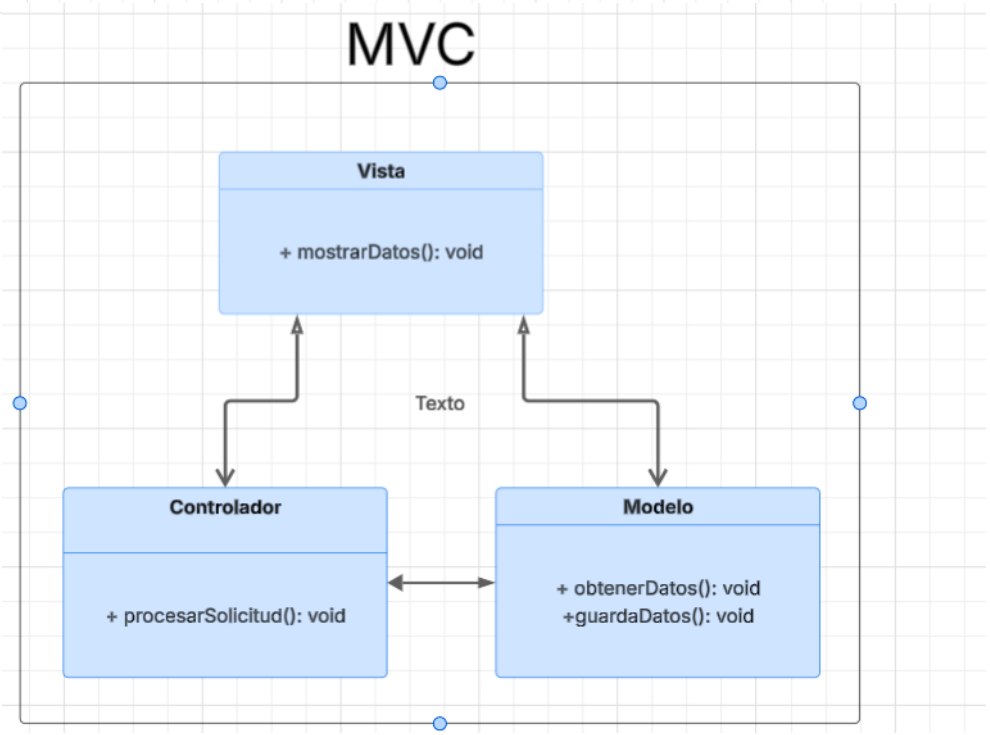
DIAGRAMA DE CLASES UML




Singleton
NotificaionService
<<Singleten>>
-instancia: NotificacionService
+ gestinstancia(): NotificaionService
+enviarNotificaion(mensaje: string): void
<<usa>>
Usario
-id: int
-nombre: string


observer
<<interface>>
Obervador
+actualizar(mensaje: string): void
notifica
Sujeto
-observadores: List<Observador>
+ agregarObervador(): void
+notificar(): void
Voluntario
+actualizar(mensaje: string): void
Veterinario
+actualizar(mensaje: string): void


Factory Method
<<Interfaz>>
UsuarioFactory
+ crearUsuario(): Usuario
Texto
ciudadanoFactory
+cerarUsuario(): Usuario
VoluntarioFactory
+ crearUsuario(): Usuario
VeternarioFactory
+ crearUsuario(): Usuario


MVC
Vista
+ mostrarDatos(): void
Texto
Controlador
+ procesarSolicitud(): void
Modelo
+ obtenerDatos(): void
+guardaDatos(): void

Arquitectura del sistema integrada al flujo de versiones.

Patrones de diseño utilizados:

Para el diseño del Sistema de Control de Rescate y Adopción Animal se utilizarán cuatro patrones de diseño que permitirán organizar mejor sus componentes y facilitar su mantenimiento.

1. Singleton

Este patrón se aplicará en el gestor central de casos para mantener una sola instancia encargada de coordinar el registro y la actualización de los rescates.

Justificación: permite centralizar la gestión de los casos y evitar la creación innecesaria de varias instancias del mismo gestor.

2. Observer

Este patrón se utilizará para notificar a los voluntarios cuando se registre un nuevo caso de rescate o se produzcan cambios importantes.

Justificación: facilita el envío de notificaciones y permite agregar nuevos destinatarios sin modificar la lógica principal del sistema.

3. Factory Method

Este patrón permitirá crear los objetos correspondientes a cada especie de animal, como perros, gatos y fauna silvestre, según el tipo seleccionado al registrar el caso.

Justificación: organiza la creación de los objetos y facilita la incorporación de nuevas especies en el futuro.

4. MVC (Modelo-Vista-Controlador)

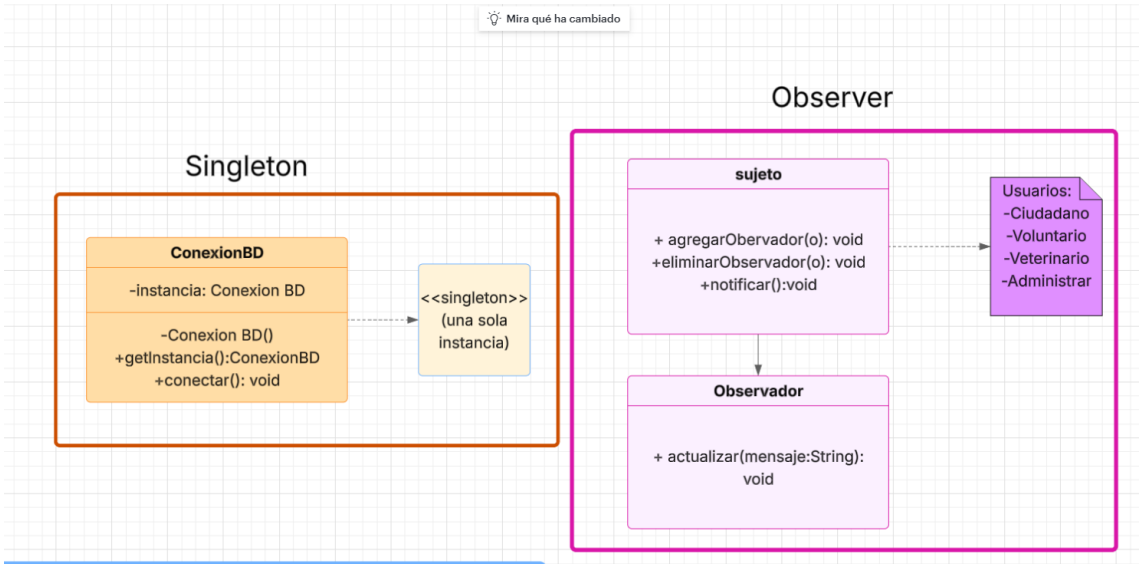
Este patrón dividirá el sistema en tres partes: el modelo, que administra la información de los animales y los rescates; la vista, que presenta los formularios y los datos al usuario; y el controlador, que recibe las solicitudes y coordina las operaciones.

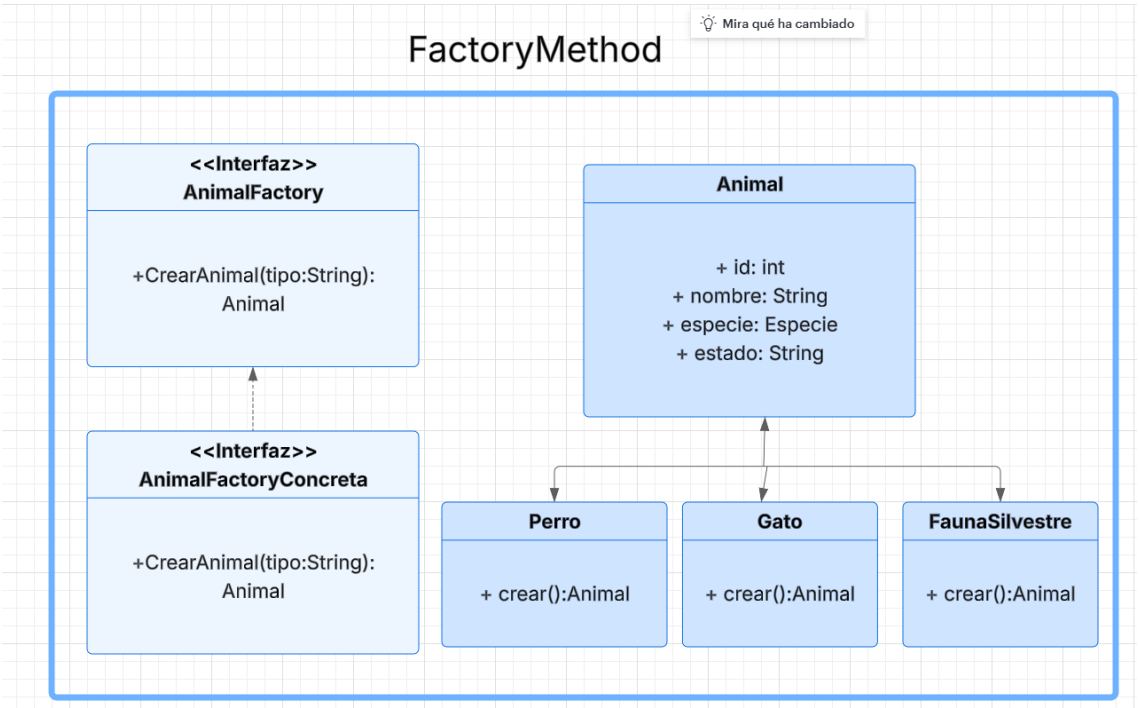
Justificación: permite separar las responsabilidades de cada componente, facilitando la comprensión y el mantenimiento del sistema.

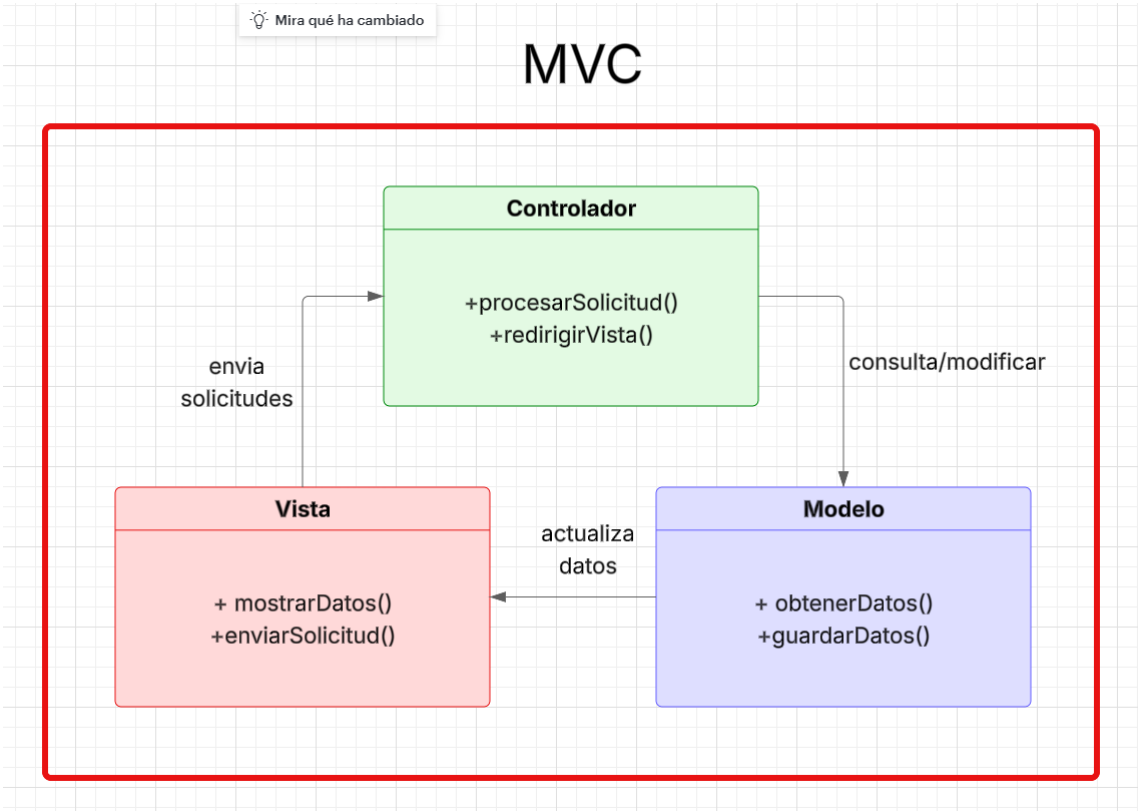
DIAGRAMA DE SECUENCIA:

El diagrama de secuencia representa el proceso de registro de un animal rescatado y muestra cómo interactúan los componentes del sistema. Primero, el ciudadano envía los datos del animal y el controlador valida la información recibida. Luego, se crea el caso de rescate y se utiliza la fábrica para generar el objeto correspondiente a la especie. Después, se asigna un veterinario responsable y el patrón Observer notifica a los usuarios interesados. Finalmente, el sistema confirma que el caso ha sido registrado correctamente.

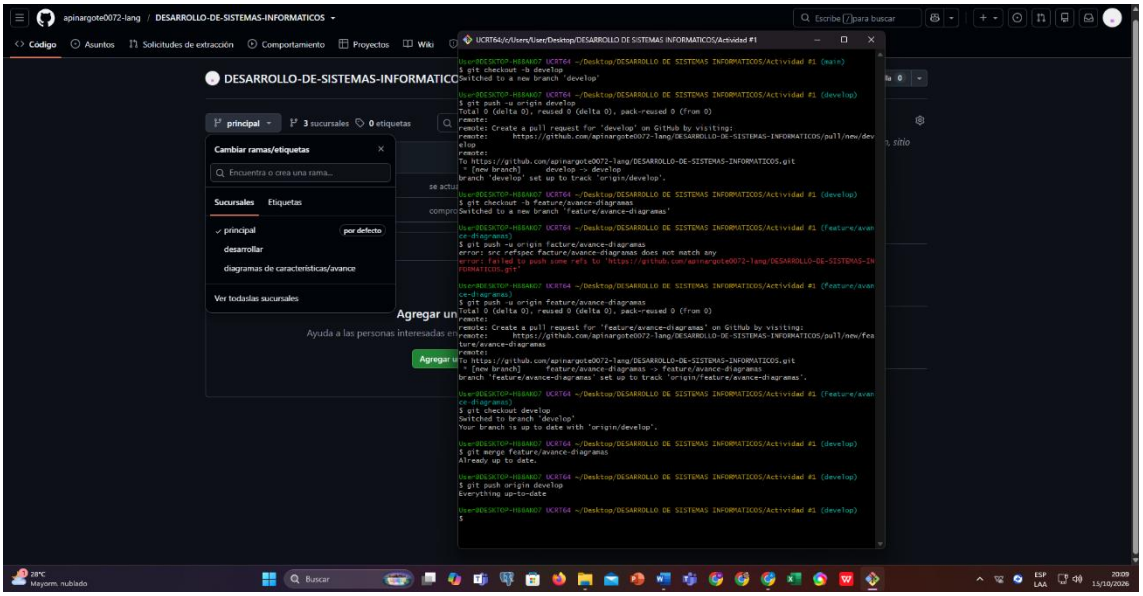
PATRONES DE DISEÑO Y DIAGRAMA DE SECUENCIA





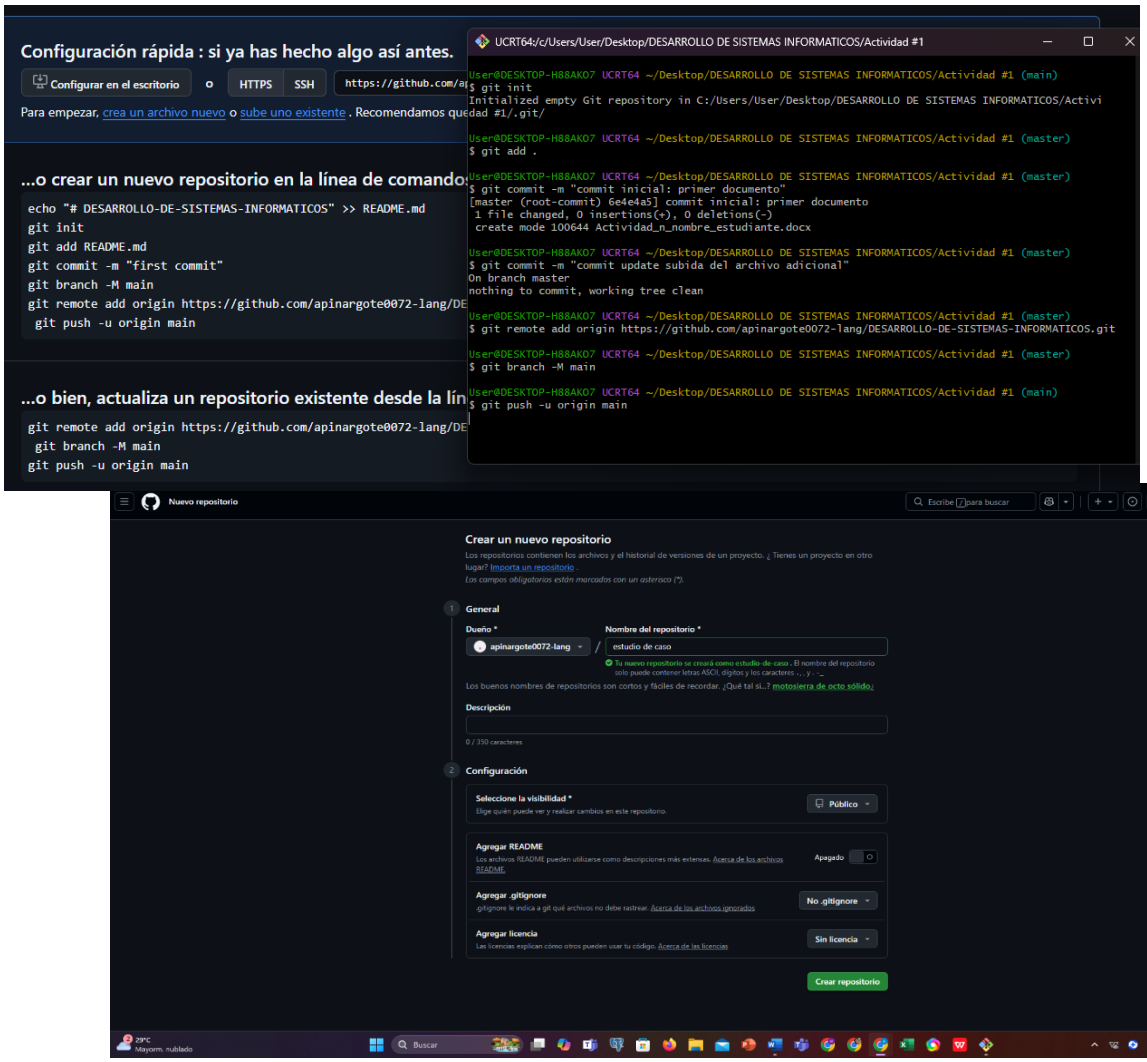


Repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

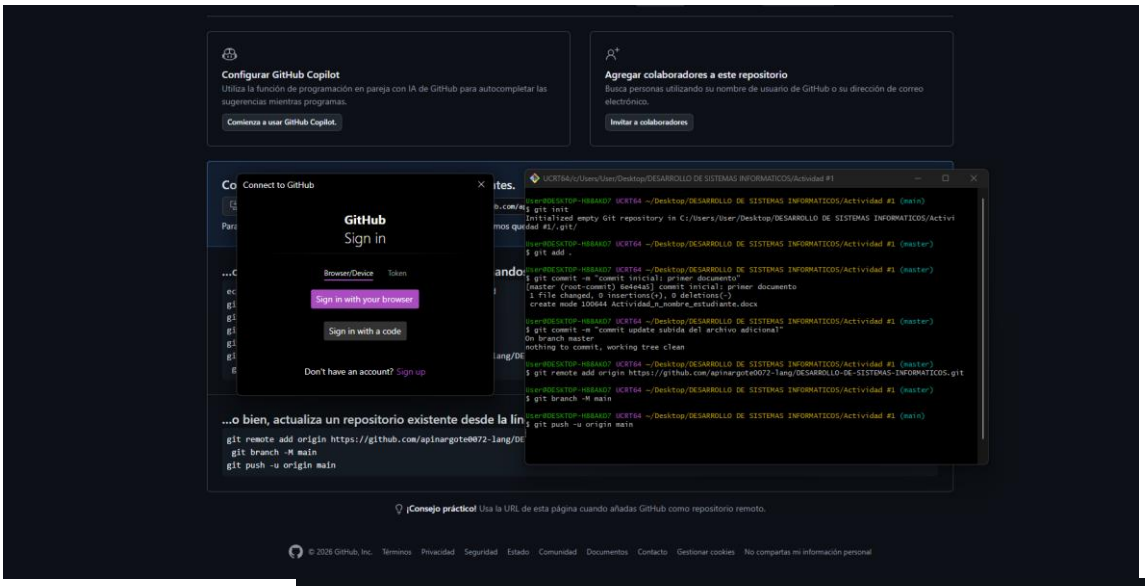


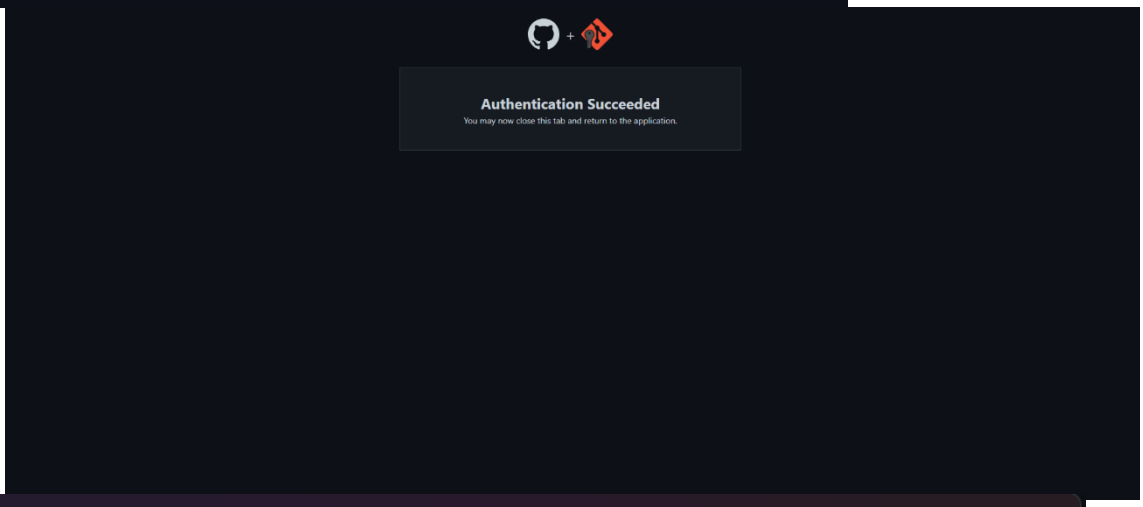
Evidencias


ración rápida : si ya has hecho algo así antes.
UCRT64:/c/Users/User/Desktop/desarrollodesistemas
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/desarrollodesistemas (master)
$ git init
Initialized empty Git repository in C:/Users/User/Desktop/desarrollodesistemas/.git/
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/desarrollodesistemas (master)
$


Configuración rápida : si ya has hecho algo así antes.
Configurar en el escritorio
o
HTTPS
SSH
Para empezar, crea un archivo nuevo o sube uno existente . Recomendamos que
...o crear un nuevo repositorio en la línea de comando
echo "# DESARROLLO-DE-SISTEMAS-INFORMATICOS" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/apinargote0072-lang/DE
git push -u origin main
...o bien, actualiza un repositorio existente desde la lín
git remote add origin https://github.com/apinargote0072-lang/DE
git branch -M main
git push -u origin main
UCRT64:/c/Users/User/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (main)
$ git init
Initialized empty Git repository in C:/Users/User/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Activi
dad #1/.git/
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git add .
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git commit -m "commit inicial: primer documento"
[master (root-commit) 6e4e4a5] commit inicial: primer documento
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Actividad_n_nombre_estudiante.docx
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git commit -m "commit update subida del archivo adicional"
On branch master
nothing to commit, working tree clean
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git remote add origin https://github.com/apinargote0072-lang/DESARROLLO-DE-SISTEMAS-INFORMATICOS.git
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git branch -M main
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (main)
$ git push -u origin main

Nuevo repositorio
Crear un nuevo repositorio
Los repositorios contienen los archivos y el historial de versiones de un proyecto. ¿ Tienes un proyecto en otro lugar? Importa un repositorio .
Los campos obligatorios están marcados con un asterisco (*).
General
Dueño *
apinargote0072-lang
Nombre del repositorio *
estudio de caso
Tu nuevo repositorio se creará como estudio-de-caso . El nombre del repositorio solo puede contener letras ASCII, dígitos y los caracteres -, . y _.
Los buenos nombres de repositorios son cortos y fáciles de recordar. ¿Qué tal si...? motosierra de octo sólido
Descripción
0 / 350 caracteres
Configuración
Seleccione la visibilidad *
Elige quién puede ver y realizar cambios en este repositorio.
Público
Agregar README
Los archivos README pueden utilizarse como descripciones más extensas. Acerca de los archivos README
Apagado
Agregar .gitignore
.gitignore le indica a git qué archivos no debe rastrear. Acerca de los archivos ignorados
No .gitignore
Agregar licencia
Las licencias explican cómo otros pueden usar tu código. Acerca de las licencias
Sin licencia
Crear repositorio






```
UCRT64:/c/Users/User/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git commit -m "commit inicial: primer documento"
[master (root-commit) 6e4e4a5] commit inicial: primer documento
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Actividad_n_nombre_estudiante.docx

User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git commit -m "commit update subida del archivo adicional"
On branch master
nothing to commit, working tree clean

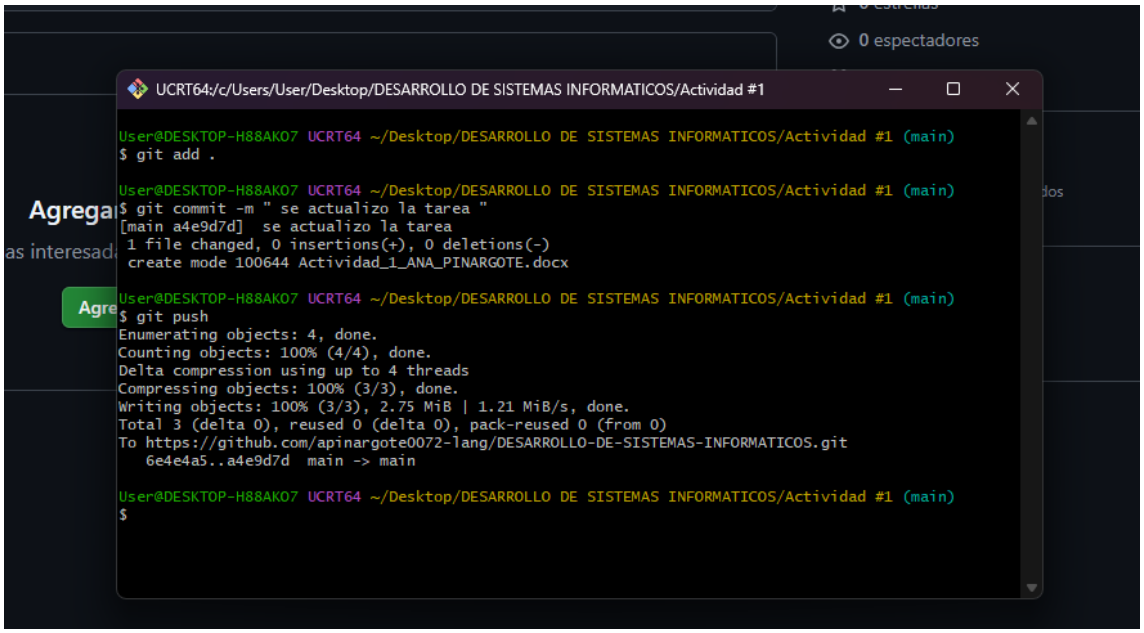
User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git remote add origin https://github.com/apinargote0072-lang/DESARROLLO-DE-SISTEMAS-INFORMATICOS.git

User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (master)
$ git branch -M main

User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.90 MiB | 895.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/apinargote0072-lang/DESARROLLO-DE-SISTEMAS-INFORMATICOS.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

User@DESKTOP-H88AKO7 UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad #1 (main)
$
```

Actualización del documento



Bibliografía

(https://en.wikipedia.org/wiki/GitHub, s.f.)

https://github.com/

https://git-scm.com/install/windows

https://lucid.app/lucidchart/c951c162-9156-40f6-8ef2-ba5680924013/edit?invitationId=inv_68541560-2ee1-44dc-8427-6368dd11ab0e&page=0_0#

LINKS DE GITHUB:

https://github.com/apinargote0072-lang/DESARROLLO-DE-SISTEMAS-INFORMATICOS